Neural Network Backpropagation

June 2, 2026

1 Backpropagation as Reverse Composition of Jacobians

2 Section 2 - Local Pullbacks

2.1 Task 2.1a — Derivation

The loss function is defined as:

$$L = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

To compute the gradient with respect to $\hat{y}$, we differentiate the squared error term.

Derivative of the squared error:

$$\frac{d}{d\hat{y}_i}(\hat{y}_i - y_i)^2 = 2(\hat{y}_i - y_i)$$

Including the factor $\frac{1}{N}$:

$$\frac{\partial L}{\partial \hat{y}_i} = \frac{2}{N}(\hat{y}_i - y_i)$$

Vector form:

$$\frac{\partial L}{\partial \hat{y}} = \frac{2}{N}(\hat{y} - y)$$

This gradient acts as the seed gradient that initiates reverse-mode differentiation.

2.2 Task 2.1b — Implementation

[1]:
```
import numpy as np

def compute_dL_dyhat(y_hat, y):

    # number of samples
    N = y_hat.shape[0]
```

```
    # compute gradient
    dL_dyhat = (2.0 / N) * (y_hat - y)

    return dL_dyhat
```

2.2.1 Automated Check

[2]:
```
import numpy as np

np.random.seed(0)

N = 7
y_hat = np.random.randn(N)
y = np.random.randn(N)

# student function
dL_dyhat = compute_dL_dyhat(y_hat, y)

# shape
assert isinstance(dL_dyhat, np.ndarray), "dL_dyhat must be a NumPy array"
assert (
    dL_dyhat.shape == y_hat.shape
), f"Shape mismatch: expected {y_hat.shape}, got {dL_dyhat.shape}"

# value (closed-form)
expected = (2.0 / N) * (y_hat - y)

assert (
    np.allclose(dL_dyhat, expected, rtol=1e-10, atol=1e-12)
), "Incorrect dL/dy_hat"

print("+++ Task 2.1b checks passed.")
```

```
+++ Task 2.1b checks passed.
```

2.3 Task 2.2a — Derivation

In the output layer, the predicted output is generated using an affine transformation:

$$\hat{y} = Wx + b$$

where W is the weight matrix, x is the input vector to the layer, and b is the bias term.

From Task 2.1, the gradient of the loss with respect to the predicted output is given by:

$$\frac{\partial L}{\partial \hat{y}}$$

This gradient acts as the starting point for backpropagation.

To propagate this gradient through the affine transformation, we apply the chain rule.

First, consider the gradient with respect to the weights W:

$$\frac{\partial L}{\partial W} = \left(\frac{\partial L}{\partial \hat{y}}\right) x^T$$

Next, the gradient with respect to the bias b is:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}}$$

Finally, the gradient with respect to the input x is:

$$\frac{\partial L}{\partial x} = W^T \left(\frac{\partial L}{\partial \hat{y}}\right)$$

These gradients describe how the error signal propagates backward through the affine layer during backpropagation.

2.4 Task 2.2b — Implementation

[3]:
```python
def backward_output_affine(dL_dyhat, H, v):

    # compute the gradient with respect to H
    dL_dH = np.outer(dL_dyhat, v)

    # compute the gradient with respect to v
    dL_dv = H.T @ dL_dyhat

    # compute the gradient with respect to the bias c
    dL_dc = np.sum(dL_dyhat)

    return dL_dH, dL_dv, dL_dc
```

2.4.1 Automated Check

[4]:
```python
def loss_from_yhat(y_hat, y):
    return np.mean((y_hat - y) ** 2)

def finite_diff_grad_H_v_c(H, v, c, y, eps=1e-6):
    N, h = H.shape

    # initialise gradients
    gH = np.zeros_like(H)
    gv = np.zeros_like(v)
    gc = 0.0
```

```
    base_yhat = H @ v + c
    base_L = loss_from_yhat(base_yhat, y)

    # gradient with respect to H
    for i in range(N):
        for j in range(h):
            H_p = H.copy()
            H_m = H.copy()
            H_p[i, j] += eps
            H_m[i, j] -= eps
            L_p = loss_from_yhat(H_p @ v + c, y)
            L_m = loss_from_yhat(H_m @ v + c, y)
            gH[i, j] = (L_p - L_m) / (2 * eps)

    # gradient with respect to v
    for j in range(h):
        v_p = v.copy()
        v_m = v.copy()
        v_p[j] += eps
        v_m[j] -= eps
        L_p = loss_from_yhat(H @ v_p + c, y)
        L_m = loss_from_yhat(H @ v_m + c, y)
        gv[j] = (L_p - L_m) / (2 * eps)

    # gradient with respect to c
    L_p = loss_from_yhat(H @ v + (c + eps), y)
    L_m = loss_from_yhat(H @ v + (c - eps), y)
    gc = (L_p - L_m) / (2 * eps)

    return gH, gv, gc


np.random.seed(1)
N, h = 5, 4
H = np.random.randn(N, h)
v = np.random.randn(h)
c = np.random.randn()
y = np.random.randn(N)

# upstream from Task 2.1
y_hat = H @ v + c
dL_dyhat = (2.0 / N) * (y_hat - y)

# student function
dL_dH, dL_dv, dL_dc = backward_output_affine(dL_dyhat, H, v)
```

```python
# shape checks
assert dL_dH.shape == H.shape, f"dL_dH shape mismatch: expected {H.shape}, got␣
 ↪{dL_dH.shape}"
assert dL_dv.shape == v.shape, f"dL_dv shape mismatch: expected {v.shape}, got␣
 ↪{dL_dv.shape}"
assert (
    np.isscalar(dL_dc)
    or (isinstance(dL_dc, np.ndarray) and dL_dc.shape == ())
), "dL_dc must be a scalar"

# value checks vs finite differences
gH_num, gv_num, gc_num = finite_diff_grad_H_v_c(H, v, c, y, eps=1e-6)

def rel_err(a, b):
    return np.linalg.norm(a - b) / (np.linalg.norm(a) + np.linalg.norm(b) +␣
 ↪1e-12)

errH = rel_err(dL_dH, gH_num)
errv = rel_err(dL_dv, gv_num)
errc = abs(dL_dc - gc_num) / (abs(dL_dc) + abs(gc_num) + 1e-12)

assert errH < 2e-5, f"dL_dH failed finite-diff check (rel err {errH:.2e})"
assert errv < 2e-5, f"dL_dv failed finite-diff check (rel err {errv:.2e})"
assert errc < 2e-5, f"dL_dc failed finite-diff check (rel err {errc:.2e})"

print("+++ Task 2.2b checks passed.")
```

```
+++ Task 2.2b checks passed.
```

2.5 Task 2.3a — Derivation

In this section, the activation function is ReLU, which is applied elementwise to the input Z:

$$H = \mathrm{ReLU}(Z)$$

The ReLU function is defined elementwise as:

$$H_{ij} = \begin{cases} Z_{ij}, & \text{if } Z_{ij} > 0 \\ 0, & \text{if } Z_{ij} \leq 0 \end{cases}$$

Since ReLU is applied elementwise, its derivative is also computed elementwise:

$$\frac{\partial H_{ij}}{\partial Z_{ij}} = \begin{cases} 1, & \text{if } Z_{ij} > 0 \\ 0, & \text{if } Z_{ij} \leq 0 \end{cases}$$

Using the chain rule, the gradient of the loss with respect to Z is obtained by multiplying the upstream gradient $\frac{\partial L}{\partial H}$ elementwise with the derivative of ReLU:

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial H} \odot \mathbf{1}(Z > 0)$$

where $\odot$ denotes elementwise multiplication and $\mathbf{1}(Z > 0)$ is an indicator function that equals 1 when $Z > 0$ and 0 otherwise.

This shows that the gradient propagates only through the entries where Z is positive.

2.6 Task 2.3b — Implementation

[5]:
```python
def backward_activation_relu(dL_dH, Z):

    # create a mask that keeps only positive values
    mask = (Z > 0)

    # pass the gradient only where Z is positive
    dL_dZ = dL_dH * mask

    return dL_dZ
```

2.6.1 Automated Check

[6]:
```python
import numpy as np

def relu(Z):
    return np.maximum(Z, 0.0)

def loss_via_relu(Z, dL_dH):
    # Synthetic scalar objective: <dL_dH, H> where H = relu(Z)
    # This makes upstream gradient exactly dL_dH.
    H = relu(Z)
    return float(np.sum(dL_dH * H))

def finite_diff_grad_Z(Z, dL_dH, eps=1e-6):
    gZ = np.zeros_like(Z)
    it = np.nditer(Z, flags=["multi_index"], op_flags=["readwrite"])
    while not it.finished:
        idx = it.multi_index
        Z_p = Z.copy()
        Z_m = Z.copy()
        Z_p[idx] += eps
        Z_m[idx] -= eps
        L_p = loss_via_relu(Z_p, dL_dH)
        L_m = loss_via_relu(Z_m, dL_dH)
        gZ[idx] = (L_p - L_m) / (2 * eps)
        it.iternext()
    return gZ
```

```python
np.random.seed(2)
Z = np.random.randn(4, 6)
dL_dH = np.random.randn(4, 6)

# student function
dL_dZ = backward_activation_relu(dL_dH, Z)

# shape
assert (
    dL_dZ.shape == Z.shape
), f"Shape mismatch: expected {Z.shape}, got {dL_dZ.shape}"

# value vs finite differences (avoid kink exactly at 0 by nudging)
Z_safe = Z.copy()
Z_safe[np.abs(Z_safe) < 1e-3] += 1e-2
dL_dZ_num = finite_diff_grad_Z(Z_safe, dL_dH, eps=1e-6)

def rel_err(a, b):
    return (
        np.linalg.norm(a - b)
        / (np.linalg.norm(a) + np.linalg.norm(b) + 1e-12)
    )

err = rel_err(dL_dZ, dL_dZ_num)
assert err < 2e-5, f"Failed finite-diff check (rel err {err:.2e})"
print("+++ Task 2.3b checks passed.")
```

```
+++ Task 2.3b checks passed.
```

2.7 Task 2.4a — Derivation

The hidden layer performs an affine transformation of the form:

$$Z = XW + b$$

where: - $X \in \mathbb{R}^{N \times d}$ is the batch of input samples,
- $W \in \mathbb{R}^{d \times h}$ is the weight matrix,
- $b \in \mathbb{R}^{h}$ is the bias vector,
- $Z \in \mathbb{R}^{N \times h}$ is the output of the layer.

Since this mapping is affine, we analyze how small perturbations affect Z. The first-order variation is given by:

$$\delta Z = \delta X\, W + X\, \delta W + \delta b$$

where δX, δW, and δb represent small changes in the input, weights, and bias, respectively.

The bias is broadcast across the batch dimension, meaning the same bias vector is added to each row of Z.

Using the perturbation formulation of the loss:

$$\delta L = \left\langle \frac{\partial L}{\partial Z}, \delta Z \right\rangle$$

Substituting δZ gives:

$$\delta L = \left\langle \frac{\partial L}{\partial Z}, \delta X W \right\rangle + \left\langle \frac{\partial L}{\partial Z}, X \delta W \right\rangle + \left\langle \frac{\partial L}{\partial Z}, \delta b \right\rangle$$

To isolate each gradient, we rearrange terms so each perturbation appears independently.

For the weights:

$$\frac{\partial L}{\partial W} = X^T \frac{\partial L}{\partial Z}$$

For the input:

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z} W^T$$

For the bias, since it is shared across all samples, the contributions accumulate across the batch:

$$\frac{\partial L}{\partial b} = \sum_{i=1}^{N} \frac{\partial L}{\partial Z_i}$$

This produces a vector in $\mathbb{R}^h$, matching the shape of b.

Therefore, the gradients of the affine layer are:

$$\frac{\partial L}{\partial W} = X^T \frac{\partial L}{\partial Z}$$

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z} W^T$$

$$\frac{\partial L}{\partial b} = \sum_{i=1}^{N} \frac{\partial L}{\partial Z_i}$$

These expressions describe how gradients are propagated backward through the affine layer during backpropagation.

2.8 Task 2.4b — Implementation

[7]:
```python
def backward_input_affine(dL_dZ, X, W):

    # compute how the loss changes with respect to the weight matrix
    dL_dW = X.T @ dL_dZ

    # compute the gradient for the bias by summing the contributions from all␣
 ↪samples in the batch
    dL_db = np.sum(dL_dZ, axis=0)

    # compute how the loss changes with respect to the input X
    dL_dX = dL_dZ @ W.T

    return dL_dW, dL_db, dL_dX
```

2.8.1 Automated Check

[8]:
```python
import numpy as np

def loss_from_Z(Z, dL_dZ):
    # Synthetic scalar objective: <dL_dZ, Z>
    # This makes upstream gradient exactly dL_dZ.
    return float(np.sum(dL_dZ * Z))

def finite_diff_grad_X_W_b(X, W, b, dL_dZ, eps=1e-6):
    N, d = X.shape
    d_, h = W.shape
    assert d_ == d

    # base
    def Z_of(X_, W_, b_):
        return X_ @ W_ + b_

    gX = np.zeros_like(X)
    gW = np.zeros_like(W)
    gb = np.zeros_like(b)

    # X
    for i in range(N):
        for j in range(d):
            X_p = X.copy()
            X_m = X.copy()
            X_p[i, j] += eps
            X_m[i, j] -= eps
            L_p = loss_from_Z(Z_of(X_p, W, b), dL_dZ)
            L_m = loss_from_Z(Z_of(X_m, W, b), dL_dZ)
```

```
            gX[i, j] = (L_p - L_m) / (2 * eps)

    # W
    for i in range(d):
        for j in range(h):
            W_p = W.copy()
            W_m = W.copy()
            W_p[i, j] += eps
            W_m[i, j] -= eps
            L_p = loss_from_Z(Z_of(X, W_p, b), dL_dZ)
            L_m = loss_from_Z(Z_of(X, W_m, b), dL_dZ)
            gW[i, j] = (L_p - L_m) / (2 * eps)

    # b
    for j in range(h):
        b_p = b.copy()
        b_m = b.copy()
        b_p[j] += eps
        b_m[j] -= eps
        L_p = loss_from_Z(Z_of(X, W, b_p), dL_dZ)
        L_m = loss_from_Z(Z_of(X, W, b_m), dL_dZ)
        gb[j] = (L_p - L_m) / (2 * eps)

    return gX, gW, gb


np.random.seed(3)
N, d, h = 4, 5, 3
X = np.random.randn(N, d)
W = np.random.randn(d, h)
b = np.random.randn(h)

# arbitrary upstream gradient
dL_dZ = np.random.randn(N, h)

# student function
dL_dW, dL_db, dL_dX = backward_input_affine(dL_dZ, X, W)

# shape checks
assert (
    dL_dW.shape == W.shape
), f"dL_dW shape mismatch: expected {W.shape}, got {dL_dW.shape}"
assert (
    dL_db.shape == b.shape
), f"dL_db shape mismatch: expected {b.shape}, got {dL_db.shape}"
assert (
    dL_dX.shape == X.shape
```

```
), f"dL_dX shape mismatch: expected {X.shape}, got {dL_dX.shape}"

# value checks vs finite differences
gX_num, gW_num, gb_num = finite_diff_grad_X_W_b(X, W, b, dL_dZ, eps=1e-6)

def rel_err(a, b):
    return (
        np.linalg.norm(a - b)
        / (np.linalg.norm(a) + np.linalg.norm(b) + 1e-12)
    )

errX = rel_err(dL_dX, gX_num)
errW = rel_err(dL_dW, gW_num)
errb = rel_err(dL_db, gb_num)

assert errX < 2e-5, f"dL_dX failed finite-diff check (rel err {errX:.2e})"
assert errW < 2e-5, f"dL_dW failed finite-diff check (rel err {errW:.2e})"
assert errb < 2e-5, f"dL_db failed finite-diff check (rel err {errb:.2e})"

print("+++ Task 2.4b checks passed.")
```

```
+++ Task 2.4b checks passed.
```

3 Section 3 — Reverse Composition

3.1 Task 3.1a — Reverse Topological Order Explanation

In a computational network, each node represents an operation that produces an output that is consumed by nodes further down the graph. During the forward pass, computations move from the input nodes to the final loss value. When computing gradients, the procedure runs in the other direction. A node can only calculate its gradient after receiving gradient information from all nodes that rely on its output. Because of this dependency, the graph must be treated in reverse topological order. This ensures that gradients from downstream nodes are available before each node calculates its own contribution. This structure allows gradient information to be effectively distributed throughout the network.

3.1.1 Task 3.1b — Implementation

[9]:
```
def full_forward_backward(X, y, W, b, v, c):

    # -- Forward pass --

    # first affine transformation
    Z = X @ W + b

    # apply ReLU activation
    H = np.maximum(Z, 0.0)
```

```
    # output layer affine transformation
    y_hat = H @ v + c

    # compute mean squared error loss
    loss = np.mean((y_hat - y) ** 2)


    # -- Backward pass (reverse order) --

    # gradient of the loss with respect to predictions
    dL_dyhat = compute_dL_dyhat(y_hat, y)

    # backpropagate through the output affine layer
    dL_dH, dL_dv, dL_dc = backward_output_affine(dL_dyhat, H, v)

    # propagate gradients through the ReLU activation
    dL_dZ = backward_activation_relu(dL_dH, Z)

    # propagate gradients through the input affine layer
    dL_dW, dL_db, dL_dX = backward_input_affine(dL_dZ, X, W)

    return loss, dL_dW, dL_db, dL_dv, dL_dc
```

3.1.2 Automated Check

[10]:
```
import numpy as np

def loss_only(X, y, W, b, v, c):
    Z = X @ W + b
    H = np.maximum(Z, 0.0)
    y_hat = H @ v + c
    return np.mean((y_hat - y) ** 2)

def rel_err(a, b):
    return (
        np.linalg.norm(a - b)
        / (np.linalg.norm(a) + np.linalg.norm(b) + 1e-12)
    )

np.random.seed(10)
N, d, h = 4, 5, 3
X = np.random.randn(N, d)
y = np.random.randn(N)
W = np.random.randn(d, h)
b = np.random.randn(h)
v = np.random.randn(h)
c = np.random.randn()
```

```
# student implementation
loss, dW, db, dv, dc = full_forward_backward(X, y, W, b, v, c)

# --- finite difference check ---
eps = 1e-6

def finite_diff_param(param, param_name):
    grad = np.zeros_like(param)
    it = np.nditer(param, flags=["multi_index"], op_flags=["readwrite"])
    while not it.finished:
        idx = it.multi_index

        params_p = dict(W=W.copy(), b=b.copy(), v=v.copy(), c=c)
        params_m = dict(W=W.copy(), b=b.copy(), v=v.copy(), c=c)

        params_p[param_name][idx] += eps
        params_m[param_name][idx] -= eps

        grad[idx] = (
            loss_only(X, y, **params_p) - loss_only(X, y, **params_m)
        ) / (2 * eps)

        it.iternext()

    return grad


# check W
dW_num = finite_diff_param(W, "W")
assert rel_err(dW, dW_num) < 2e-5, "dW failed finite-difference check"

# check b
db_num = finite_diff_param(b, "b")
assert rel_err(db, db_num) < 2e-5, "db failed finite-difference check"

# check v
dv_num = finite_diff_param(v, "v")
assert rel_err(dv, dv_num) < 2e-5, "dv failed finite-difference check"

# check c
c_p = c + eps
c_m = c - eps
dc_num = (
    loss_only(X, y, W, b, v, c_p) - loss_only(X, y, W, b, v, c_m)
) / (2 * eps)
```

```
assert (
    abs(dc - dc_num) / (abs(dc) + abs(dc_num) + 1e-12) < 2e-5
), "dc failed finite-difference check"

print("+++ Task 3.1b Full reverse composition checks passed.")
```

```
+++ Task 3.1b Full reverse composition checks passed.
```

4 Section 4 — Gradient Verification

4.1 Task 4.1a — Conceptual Understanding

Finite differences are used to validate gradients by numerically calculating how the loss varies as a parameter is significantly disturbed. This method does not produce gradients since it just evaluates the function at close places rather than studying the entire graph or using the chain rule. Analytical gradients are obtained instead by backpropagation.

Central difference is preferable to forward difference because it assesses the function on both sides of the parameter value. Using information from both directions results in a more accurate approximation and lowers numerical inaccuracy.

Because gradient magnitudes can vary greatly, relative error is preferred over absolute error. Relative error scales the difference, so the comparison is meaningful regardless of gradient magnitude.

4.2 Task 4.1b — Implementation

[11]:
```
def gradient_check(X, y, W, b, v, c, param_name):

    # compute analytical gradients from the full forward and backward pass
    loss, dW, db, dv, dc = full_forward_backward(X, y, W, b, v, c)

    # choose the analytical gradient for the requested parameter
    if param_name == "W":
        analytic_grad = dW
        param = W
    elif param_name == "b":
        analytic_grad = db
        param = b
    elif param_name == "v":
        analytic_grad = dv
        param = v
    elif param_name == "c":
        analytic_grad = dc
        param = c
    else:
        raise ValueError("param_name must be one of: 'W', 'b', 'v', or 'c'")

    eps = 1e-6
```

```
    # handle the scalar parameter c separately
    if np.isscalar(param):
        c_p = c + eps
        c_m = c - eps

        loss_p, _, _, _, _ = full_forward_backward(X, y, W, b, v, c_p)
        loss_m, _, _, _, _ = full_forward_backward(X, y, W, b, v, c_m)

        numeric_grad = (loss_p - loss_m) / (2 * eps)

        rel_error = abs(analytic_grad - numeric_grad) / (
            abs(analytic_grad) + abs(numeric_grad) + 1e-12
        )

        return rel_error

    # compute numerical gradients for tensor parameters using central␣
↪differences
    numeric_grad = np.zeros_like(param)
    it = np.nditer(param, flags=["multi_index"], op_flags=["readwrite"])

    while not it.finished:
        idx = it.multi_index

        W_p, b_p, v_p, c_p = W.copy(), b.copy(), v.copy(), c
        W_m, b_m, v_m, c_m = W.copy(), b.copy(), v.copy(), c

        if param_name == "W":
            W_p[idx] += eps
            W_m[idx] -= eps
        elif param_name == "b":
            b_p[idx] += eps
            b_m[idx] -= eps
        elif param_name == "v":
            v_p[idx] += eps
            v_m[idx] -= eps

        loss_p, _, _, _, _ = full_forward_backward(X, y, W_p, b_p, v_p, c_p)
        loss_m, _, _, _, _ = full_forward_backward(X, y, W_m, b_m, v_m, c_m)

        numeric_grad[idx] = (loss_p - loss_m) / (2 * eps)
        it.iternext()

    # compute relative error between analytical and numerical gradients
    rel_error = np.linalg.norm(analytic_grad - numeric_grad) / (
        np.linalg.norm(analytic_grad) + np.linalg.norm(numeric_grad) + 1e-12
    )
```

```
    return rel_error
```

4.2.1 Automated Check

[12]:
```
import numpy as np

np.random.seed(42)
N, d, h = 3, 4, 2
X = np.random.randn(N, d)
y = np.random.randn(N)
W = np.random.randn(d, h)
b = np.random.randn(h)
v = np.random.randn(h)
c = np.random.randn()

rel_error = gradient_check(X, y, W, b, v, c, param_name="W")

assert (
    rel_error < 2e-5
), f"Gradient check failed (rel error {rel_error:.2e})"

print("+++ Task 4.1b Gradient checking passed.")
```

```
+++ Task 4.1b Gradient checking passed.
```

5 Section 5 — Conceptual Reflection

5.1 Task 5.1 — Reverse-Mode Efficiency

Reverse-mode differentiation is effective for neural networks since the loss function is scalar. Instead of computing derivatives for each parameter individually, reverse-mode sends a single upstream gradient from the loss back through the network. At each layer, it uses a vector-Jacobian product to efficiently aggregate gradient contributions for all parameters in a single backward pass.

In contrast, forward-mode differentiation calculates Jacobian-vector products. To acquire the entire gradient with respect to many parameters, forward-mode would need to propagate a distinct basis vector in each parameter direction. This means that the number of passes varies with the number of parameters.

Because deep learning models frequently have millions of parameters yet output a single scalar loss, reverse-mode differentiation performs substantially better computationally. It enables the complete gradient to be computed at a cost roughly equivalent to one forward and one backward pass.

5.2 Task 5.2 — Why Order Reverses

Consider a function composed of multiple mappings:

$$f = f_L \circ \cdots \circ f_1$$

By the chain rule, the Jacobian of the composition is the product of the individual Jacobians:

$$Df = Df_L \cdots Df_1$$

When computing gradients of a scalar-valued loss, reverse-mode differentiation works with the transpose of this Jacobian. Using the transpose identity:

$$(AB)^\top = B^\top A^\top$$

the transpose of the composed Jacobian becomes:

$$(Df)^\top = Df_1^\top \cdots Df_L^\top$$

This reverses the order of the linear maps. As a result, reverse-mode applies vector–Jacobian products sequentially from the final layer back to the first.

Since the loss is scalar-valued, this approach computes the full gradient efficiently in a single backward pass.

5.3 Task 5.3 — Conceptual Challenge

The most difficult aspect of this task was determining why transposes happen during backpropagation. At first, it was unclear why gradients propagated through the transpose of weight matrices rather than the matrices themselves. After going through the derivations, it became evident that transposes form when linear maps traverse inner products while isolating perturbations. Another challenge was understanding why bias gradients must be summed over the batch dimension. Because the same bias vector is applied to each sample, the gradient contributions from all samples must be combined. Finally, tracking tensor forms during each stage of the backward pass needed close attention to guarantee that matrix multiplications and gradient dimensions were consistent.

[12]: